

SHORT REPORT

THE EFFECTS OF AI-ASSISTED SOFTWARE ENGINEERING ON CODE QUALITY AND SECURITY

A report submitted for the course

ADVANCED SOFTWARE ENGINEERING 2
(Frühlingssemester 2026)

presented by

TEAM 08

Dao Trong-Nghia (daotro01)
Mata Marco (matamar)
Kott Jan (kottjan1)
Galindo Lopez Pablo Emiliano (galinpab)

submitted in Frühlingssemester 2026
at ZHAW Zurich

May 6, 2026

Table of Contents

CHAPTER 1:	INTRODUCTION	1
1.1	Motivation	1
1.2	Challenge 1: Quality and Security Drift	1
1.3	Challenge 2: Review Cannot Keep Up	1
CHAPTER 2:	DISCUSSION AND CASE STUDY	2
2.1	Problem Framing	2
2.2	Case-Study Design	2
2.3	Structural Comparison	3
CHAPTER 3:	RESULTS AND INTERPRETATION	3
3.1	SonarQube Scan Findings	3
3.2	Interpretation	4
3.3	Lessons	4
CHAPTER 4:	LIMITATIONS	5
CHAPTER 5:	CONCLUSION	5

1 Introduction

1.1 Motivation

In this short-report we study how AI coding assistants affect code quality and security in real projects. AI made writing code cheap, but checking it is not. A large share of AI-generated code still has high-severity vulnerabilities, and that has not significantly improved as the tools get better [1].

Adoption is already near universal. The Google DORA 2025 report puts it at 90% of software professionals, with a median of two hours a day, and finds a negative correlation with software delivery stability [9]. A peer-reviewed study of 517 programming questions to ChatGPT found that 52% of its answers contained incorrect information, yet users still preferred them 35% of the time and missed the errors in 39% of cases [10].

Teams that adopt AI ship more code than reviewer capacity can handle, so the bottleneck is moving from writing to reviewing [2]. This report frames the two challenges with industry cases, then tests the quality-debt claim on a like-for-like AI-versus-human implementation under SonarQube.

1.2 Challenge 1: Quality and Security Drift

What AI puts into the codebase is buggier and more duplicated than what it replaced [1, 3]. We found two independent cases in industry.

Failure case: Clawdbot / OpenClaw. Clawdbot / OpenClaw was a viral open-source AI-agent project. During a forced rename, crypto scammers seized the GitHub organisation in ten seconds. Running instances were found exposed via Shodan with plaintext credentials, and viral growth meant nobody had time to fix it. Every prevention tool that should have caught this already existed, but none operated at the speed required [4, 5].

Working case: GitHub Dependabot at scale. GitHub Octoverse 2025 says that across 2.66 million repositories with Dependabot enabled, automated dependency-vulnerability fix times collapsed from 200–300 days down to 30–50 days [6]. This is a working example of automation closing a security gap at scale.

1.3 Challenge 2: Review Cannot Keep Up

These quality problems get worse because review cannot keep up. When bad code is produced, the people meant to catch it are buried in volume [2]. Open-source maintainers are hit hardest, and this is how the bad code reaches production. We found two independent cases in industry.

Failure case: curl bug bounty closure. curl had a bug-report program where people could report security bugs for a reward. The program got flooded with AI-generated reports that sounded professional but were almost all wrong. The maintainer tried adding stricter rules and examples of what not to submit, but nothing helped, so he shut the program down entirely [2, 7].

Working case: Google’s internal review tooling. Google’s code-review tooling reports 97% engineer satisfaction. Google has layered machine learning on top: an AutoCommenter trained on around 3 billion examples to flag best-practice violations, and a critic feature in the Jules coding

assistant that critiques AI-generated code while it is being produced [8].

2 Discussion and Case Study

2.1 Problem Framing

Both working cases only cover part of their challenge. Dependabot fixes known dependency vulnerabilities, but the Clawdbot case concerned exposed credentials and open interfaces, which are things no dependency scanner would catch. Google’s review tooling shows what automation can do, but it depends on Google’s scale, something a one-person project like curl cannot replicate. No single solution covers a whole challenge on its own, and the ones that exist require resources the projects most at risk do not have.

Research Question

Does the quality and security debt reported in the literature show up on a like-for-like AI-versus-human implementation of the same specification under one static analyser?

2.2 Case-Study Design

To probe the gaps above, we take the same specification and produce two implementations, then apply one static analysis tool to both and compare findings. This directly tests Challenge 1, quality drift. Challenge 2, the review bottleneck, is addressed indirectly through the production-versus-verification asymmetry the comparison reveals.

Specification. PM1 Project 3 is a first-semester ZHAW Java assignment: a text version of Settlers of Catan. Scope includes the base game, a cities extension via inheritance, and a random-victim robber variant. The assignment contains 30 points in total.

Two implementations. The human side is a frozen snapshot of a public first-semester student team’s implementation, `boostvolt/zhaw-catan`, used with the author’s permission. The AI side is generated by Claude Opus from a consolidated English version of the original specification, with the same scope, the same fixed helper library `ch.zhaw.hexboard`, and the same first-semester Java subset. Both sides build as a Maven project and carry the three required `requirement*` unit tests named in the original assignment.

Tool selection. The tool-selection process used two phases. First, each candidate from GitHub’s security stack was run against a Node.js benchmark project built to trigger it: an outdated CVE-bearing dependency for Dependabot and Dependency Review, a hardcoded API key for Secret Protection, a coding bug for CodeQL, and an open pull request for Copilot code review. The benchmark confirmed that each tool works in its native habitat and that the Catan study sits outside it: two static Java snapshots, no pull request, no secrets, and almost no dependencies.

SonarQube community edition, run locally against the Maven build, was selected for the Catan run. It is a static code analyser with first-class Java and Maven support that exposes bugs, reliability, maintainability, and security hotspots in a single scan without needing a pull-request diff or a dependency-change event. CodeQL was the runner-up. SonarQube won on broader signal because this study is about quality debt, not just vulnerabilities. The Node.js benchmark justifies the tool choice; Catan tests the literature claim that AI-generated code carries more quality and security debt than human-written code of equivalent scope.

Tool	Surface	Fit for Catan	Verdict
Dependabot	Manifests, lockfiles	Tiny dependency surface; nothing to flag	Discarded
GitHub Advanced Security / CodeQL	Source code	Strong on Java but security-only; misses maintainability and reliability signal	Runner-up
Secret Protection	Commits, history	Offline game, no secrets	Discarded
Copilot code review	Pull-request diff	No active pull request	Discarded
Dependency Review	Pull-request manifest diff	No pull request and no manifest change	Discarded
SonarQube	Full Java Maven project	Bugs, reliability, maintainability, and hotspots in one scan	Selected

Table 2.1: Tool-selection comparison for the Catan case study

2.3 Structural Comparison

The AI version is materially smaller than the human version at the same specification. Main source is 1,108 lines across nine files against the human implementation’s 2,718 lines across seventeen files. Test source is 260 lines in a single test file against 2,060 lines across twelve files, or 13% of the human test-source size. The AI omits a `Road` class, representing roads as two-character strings on hex edges. It also omits an abstract `Structure` base and concentrates most logic in a single `SiedlerGame` class of 435 lines, or 39% of main source. Claude Opus produced this runnable 30-point implementation in roughly ten minutes of wall-clock time; the human repository’s commit history spans weeks. The asymmetry in production speed relative to verification surface, with a test-to-main LOC ratio of 0.23 against 0.76, is the motivating gap shown in miniature: AI-assisted production is much cheaper than AI-assisted verification.

Implementation	Main LOC	Main files	Test LOC	Test/Main Ratio
Human implementation	2,718	17	2,060	0.76
AI implementation	1,108	9	260	0.23

Table 2.2: Pre-scan structural comparison of the Catan implementations

3 Results and Interpretation

3.1 SonarQube Scan Findings

SonarQube was run against both codebases on 2026-05-01. Both projects passed the configured quality gate and reported zero direct security issues. The signal is on reliability and maintainability.

Metric	AI	Human
Lines of code	1.6k	2.0k
Security rating / issues	A / 0	A / 0
Reliability rating / issues	D / 3	C / 1
Maintainability rating / issues	A / 16	A / 9
Security hotspots	3	1
Coverage	0.0%	0.0%
Duplications	0.0%	0.0%

Table 3.1: SonarQube scan findings for the AI and human Catan implementations

Coverage reads 0% on both because neither Maven build emits a JaCoCo report; the comparison here is on static signal only. Normalised by SonarQube’s own LOC figure, the AI version carries about 10.0 maintainability issues per kLOC against the human implementation’s 4.5. On reliability the gap is wider: 1.9 issues per kLOC against 0.5.

The four security hotspots across both projects are all the same rule, `java:S2245`, weak PRNG. For an offline Catan game, these are almost certainly benign uses of `Random`, not cryptographic risk.

The AI implementation’s reliability findings concentrate in `SiedlerGame`: two of three are the same “save and re-use this `Random`” anti-pattern, and the third sits in the shared `ch.zhaw.hexboard` library, which is also reported on the human side. Net of that shared library, the AI implementation carries two unique reliability issues to the human implementation’s zero.

Its maintainability list is dominated by “Brain Method” cognitive-complexity findings on `App.java`, with a score of 33, and `SiedlerGame`, with a score up to 49 against an allowed 15, plus repeated `Map-to-EnumMap` suggestions. The human side has no cognitive-complexity findings; its maintainability list is mostly style, deprecation hygiene, and small API-conformance items.

3.2 Interpretation

The scan partially supports the literature claim: quality drift is visible, but the security delta is effectively zero.

Supported on reliability and maintainability. The AI implementation scores worse on both axes, even on the per-kLOC denominator that favours the smaller codebase. The findings concentrate exactly where the structural comparison predicted: in oversized methods inside `SiedlerGame`, the same flat-package, god-class shape that the “code clones / missing abstraction” thread in the literature would lead us to expect.

Not supported on direct security. Both projects are effectively blank from a vulnerability perspective. That is more a property of the input than of the AI. Small offline Java student projects with tiny dependency surfaces and no real secret-handling path do not expose much for vulnerability-oriented tooling to bite on. CodeQL would likely report similarly little.

Caveats. Scan surface is not the same on both sides: the AI side has a test-to-main LOC ratio of 0.23 against 0.76, omits the `Road` class and the abstract `Structure` base, and ships one test file against twelve. A raw finding count is not a like-for-like comparison, which is why the density reading above matters more than the absolute totals. Separately, SonarQube does not validate game-rule correctness. The five self-flagged rule-interpretation uncertainties from the AI side sit outside what static analysis can see.

3.3 Lessons

On this assignment, SonarQube sees more static bug and code-smell pressure in the AI version, but no meaningful security delta. That maps onto Challenge 1, quality drift. Even when an AI assistant produces a runnable, specification-passing artefact in ten minutes against weeks of human work, the static-analysis fingerprint shows the cost: concentrated complexity in core logic, less abstraction depth, and a thinner self-validating test surface. Challenge 2, review bottleneck, is not directly testable in a single-artefact study, but the production-versus-verification asymmetry is the same asymmetry the curl and OpenClaw cases describe at scale.

Lesson	Meaning
Production cost decouples from verification cost	Ten minutes for 1,108 main lines shows that the cheap thing got cheaper; the expensive thing did not.
Static analysis catches quality drift in core logic	Cognitive complexity 49 in <code>SiedlerGame</code> and repeated god-class smells are visible, but rule-correctness remains outside the scanner.
A clean security rating depends on scan surface	A small offline project with no secrets, no network surface, and minimal dependencies gives vulnerability tooling little to detect.
Prevention tooling is not enough by itself	Every failure case had relevant prevention tooling somewhere in the ecosystem, but it did not operate at the speed or scale the most exposed projects needed.

Table 3.2: Main lessons from the case study

4 Limitations

This study uses a single specification, a single human team, a single AI model, a single prompt, and a single static-analysis tool. The findings are therefore suggestive, not generalisable. A fuller study would require multiple prompts, multiple models, multiple specifications, and multiple human baselines.

A second limitation is that the human and AI implementations are not perfectly equivalent artefacts. The human implementation was produced by a student team over time, while the AI implementation was generated rapidly from a consolidated English specification. The resulting codebases differ in size, abstraction structure, and test volume. This difference is analytically useful because it reveals realistic engineering-style differences, but it also means that raw metrics must be interpreted carefully.

A third limitation is tool scope. SonarQube can report static bug, reliability, maintainability, and security-hotspot signals, but it does not decide whether the game rules are correctly implemented, whether the specification was interpreted faithfully, or whether the abstractions are educationally appropriate for a first-semester Java assignment. Therefore, SonarQube findings should be treated as evidence about detectable static risks, not as a complete judgment of code quality or security.

5 Conclusion

AI-assisted software engineering is best understood as a productivity amplifier under constraints. It can make first-draft production much faster, especially where the task is bounded and conventional. However, it does not remove the need for engineering judgment. In this case study, the AI implementation was much faster to produce and materially smaller than the human implementation, but it also showed higher static reliability and maintainability issue density under SonarQube.

The security result was different. SonarQube reported no direct security issues in either project. This does not prove that AI-generated code is safe. It shows that a small offline Java assignment with almost no dependency, secret, or network surface does not provide much for vulnerability-oriented analysis to detect.

The main lesson is that AI changes the distribution of engineering work. Less effort may be needed for initial code production, but more discipline is required in review, validation, static analysis, security assessment, and architectural judgment. In this setting, AI assistance should therefore be treated as a drafting accelerator, not as a replacement for software-engineering responsibility.

Bibliography

- [1] Veracode, “2025 GenAI Code Security Report,” 2025. [Online]. Available: <https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/>
- [2] LeadDev, “Open Source Has a Big AI Slop Problem,” 2025. [Online]. Available: <https://leaddev.com/software-quality/open-source-has-a-big-ai-slop-problem>
- [3] GitClear, “AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones,” 2025. [Online]. Available: https://www.gitclear.com/ai_assistant_code_quality_2025_research
- [4] Palo Alto Networks Unit 42, “OpenClaw May Signal the Next AI Security Crisis,” 2026. [Online]. Available: <https://www.paloaltonetworks.com/blog/network-security/why-moltbot-may-signal-ai-crisis/>
- [5] P. G. Sivaram, “From Clawdbot to Moltbot,” DEV Community, 2026. [Online]. Available: <https://dev.to/sivarampg/from-clawdbot-to-moltbot-how-a-cd-crypto-scammers-and-10-seconds-of-chaos-took-down-the-4eck>
- [6] GitHub, “Octoverse 2025,” 2025. [Online]. Available: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>
- [7] The New Stack, “96% of Codebases Rely on Open Source, and AI Slop Is Putting Them at Risk,” 2025. [Online]. Available: <https://thenewstack.io/ai-slop-open-source/>
- [8] Engineer’s Codex, “How Google Takes the Pain Out of Code Reviews, with 97% Dev Satisfaction,” 2025. [Online]. Available: <https://read.engineerscodex.com/p/how-google-takes-the-pain-out-of>
- [9] Google Cloud DORA, “2025 DORA State of AI-Assisted Software Development,” 2025. [Online]. Available: <https://dora.dev/research/2025/dora-report/>
- [10] S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang, “Is Stack Overflow Obsolete? An Empirical Study of the Characteristics of ChatGPT Answers to Stack Overflow Questions,” *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024. [Online]. Available: <https://arxiv.org/pdf/2308.02312>